

PROMPT: Groups 7-16 (APIs 21-62) — TechTonicEcommerce_Nhom2

[0] ĐỌC CODE TRƯỚC (bắt buộc)

bash

```
find . -type f \( -name "*.js" -o -name "*.ts" \) | grep -v node_modules | sort
cat package.json
cat src/index.js 2>/dev/null || cat src/app.js 2>/dev/null || cat src/server.js
ls src/routes/ && cat src/routes/*.js 2>/dev/null | head -200
cat src/controllers/*.controller.js 2>/dev/null | head -200
cat src/config/db.js 2>/dev/null || cat src/database/connection.js
cat src/utils/response.js 2>/dev/null || cat src/helpers/response.js
cat src/middlewares/auth.middleware.js 2>/dev/null || cat src/middlewares/auth.
```

→ Ghi nhớ: framework, ORM/raw-query, response format, auth middleware, file naming convention.

[1] CONVENTIONS (áp dụng toàn bộ)

- **Response:** `{success:bool, data?, message?, pagination?}`
- **Pagination:** params `page`(def:1) + `limit`(def:20) → `{current_page, per_page, total_items, total_pages}`
- **Auth:** Bearer JWT → `req.user = {user_id, roles:string[]}`
- **Seller store:** `SELECT store_id FROM user_stores WHERE user_id=@uid AND store_member_role='OWNER' AND is_active=1`
- **Soft delete:** set `deleted_at=SYSUTCDATETIME()` hoặc `status='INACTIVE'`
- **SQL Server:** dùng `@param`, `OFFSET x ROWS FETCH NEXT y ROWS ONLY`
- **Comment code:** tối thiểu — chỉ comment logic không tự giải thích được

GROUP 7: Products - Seller (21-24)

| **Auth:** Seller | Verify product thuộc store mình trước khi sửa/xóa

21 • Create Product `POST /api/v1/products`

Body: `{category_id, sku, product_name, description?, price, stock_quantity?, image_urls?:[]}`

- `store_id` từ token, không nhận từ body
- `slug` = slugify(product_name) + `-` + nanoid(6)
- Check sku unique per store: `WHERE store_id=@sid AND sku=@sku`
- INSERT product → INSERT product_images (index 0: `is_primary=1`) → `UPDATE stores SET total_products+=1`

→ **201**: `{product_id, product_name, slug, price, stock_quantity, status, images[]}` → **409** SKU trùng | **400** validation

22 • Update Product `PATCH /api/v1/products/:id`

Body (partial): `{category_id?, product_name?, description?, price?, stock_quantity?, status?(ACTIVE|INACTIVE)}`

- Partial update: chỉ SET field được gửi
- Đổi `product_name` → cập nhật slug mới
- `UPDATE products SET updated_at=SYSUTCDATETIME()`

→ **200**: product object | **403** không phải owner | **404**

23 • Delete Product `DELETE /api/v1/products/:id`

- `UPDATE products SET status='INACTIVE', deleted_at=SYSUTCDATETIME()`
- `UPDATE stores SET total_products = total_products-1 WHERE total_products > 0`

→ **200**: `{message:"Đã ẩn sản phẩm"}` | **403** | **404**

24 • Get My Store Products `GET /api/v1/seller/products`

Params: `status?`, `keyword?`, `page`, `limit`

```
sql
SELECT ... FROM products
WHERE store_id=@sid AND deleted_at IS NULL
[AND status=@s] [AND product_name LIKE @kw]
ORDER BY created_at DESC
```

→ **200**: `{products[], pagination}`

GROUP 8: Store (25-28)

25 • Get Store Info (GET /api/v1/stores/:id) | Public

sql

```
SELECT s.*, COUNT(p.product_id) AS product_count
FROM stores s
LEFT JOIN products p ON p.store_id=s.store_id AND p.status='ACTIVE' AND p.deleted_at IS NULL
WHERE s.store_id=@id AND s.status='ACTIVE' AND s.deleted_at IS NULL
GROUP BY s.store_id, ...
```

→ **200**: {store_id, store_name, slug, logo_url, description, province, contact_phone, total_products, status} → **404**

26 • Get My Store (GET /api/v1/seller/store) | Seller

sql

```
SELECT s.* FROM stores s
JOIN user_stores us ON us.store_id=s.store_id
WHERE us.user_id=@uid AND us.store_member_role='OWNER' AND us.is_active=1
```

→ **200**: full store object | **404** chưa tạo shop

27 • Create Store (POST /api/v1/stores) | Seller

Body: {store_name, description?, logo_url?, contact_email?, contact_phone?, address_line?, ward?, district?, province?}

- Check chưa có shop (query user_stores) → nếu có → **409**
- `store_code` = `SHOP-` + nanoid(8).toUpperCase()
- `slug` = slugify(store_name) + `-` + nanoid(6)
- INSERT stores → INSERT user_stores (role='OWNER', is_active=1)

→ **201**: store object | **409** "Bạn đã có shop"

28 • Update Store (PATCH /api/v1/seller/store) | Seller

Body (partial): {store_name?, description?, logo_url?, contact_email?, contact_phone?, address_line?, ward?, district?, province?}

- Partial update; đổi `store_name` → cập nhật slug

→ **200**: store object

GROUP 9: Cart (29-33)

Auth: Customer | `customer_id` từ token | Auto-create cart nếu chưa có: `INSERT INTO carts(customer_id) → cart_id`

29 • Get My Cart `GET /api/v1/cart`

sql

```
SELECT ci.cart_item_id, ci.product_id, ci.quantity, ci.price_at_added,
       p.product_name, p.price AS current_price, p.stock_quantity, p.status AS
       img.image_url AS primary_image
FROM cart_items ci
JOIN products p ON ci.product_id=p.product_id
LEFT JOIN product_images img ON img.product_id=p.product_id AND img.is_primary=
WHERE ci.cart_id=@cart_id
```

- Flag `price_changed:true` nếu `p.price != ci.price_at_added`

→ **200**: `{cart_id, items[], total_amount, has_price_change:bool}`

30 • Add Item to Cart `POST /api/v1/cart/items`

Body: `{product_id, quantity}`

- Check product: `status='ACTIVE'`, `deleted_at IS NULL`, `stock_quantity >= quantity`
- Đã có trong cart: `UPDATE cart_items SET quantity+=@qty` (re-check stock sau cộng)
- Chưa có: `INSERT` với `price_at_added = p.price`

→ **200/201**: cart_item | **400** không đủ hàng | **404** sản phẩm

31 • Update Cart Item Qty `PATCH /api/v1/cart/items/:id`

Body: `{quantity}` (≥ 1)

- Verify `cart_item.cart_id = user's cart_id`
- Check `stock_quantity >= quantity`

→ **200**: cart_item | **400** | **404**

32 • Remove Cart Item `DELETE /api/v1/cart/items/:id`

- Verify ownership → `DELETE FROM cart_items WHERE cart_item_id=@id AND cart_id=@user_cart_id`

→ **200**: `{message:"Đã xóa"}` | **404**

33 • Clear Cart `DELETE /api/v1/cart`

- `DELETE FROM cart_items WHERE cart_id=@cart_id`

→ **200**: `{message:"Đã xóa giỏ hàng"}`

GROUP 10: Orders - Customer (34-37)

34 • Place Order `POST /api/v1/orders` ⚠ TRANSACTION

Body: `{store_id, items:[{product_id,quantity}], recipient_name, recipient_phone, shipping_address_line, shipping_province, shipping_ward?, shipping_district?, payment_method(COD|BANK_TRANSFER), customer_note?, address_id?, voucher_code?}`

Transaction steps:

- Verify tất cả `product_id` thuộc `store_id`, `status='ACTIVE'`
- Check `stock_quantity >= quantity` từng product
- `subtotal = SUM(price * qty)`
- Nếu `voucher_code`: validate → tính `discount_amount` (check `min_order_amount`, `usage_limit`, `ends_at >= NOW()`, per-customer limit)
- `total_amount = subtotal - discount + shipping_fee(=0 mặc định)`
- INSERT orders (`order_code='ORD-{yyyyMMdd}-{nanoid(8)}'`, `order_status='PENDING'`)
- INSERT order_items (`product_name_snapshot`, `product_image_url_snapshot`, `unit_price=p.price`)
- `UPDATE products SET stock_quantity-=qty, sold_quantity+=qty`
- INSERT order_status_histories (`prev=NULL`, `new='PENDING'`, `changed_by=@uid`)
- Nếu voucher: INSERT order_vouchers; `UPDATE vouchers SET used_count+=1`
- Xóa cart_items tương ứng (nếu đặt từ giỏ hàng)

→ **201**: `{order_id, order_code, order_status:'PENDING', total_amount}` → **400** không đủ hàng | **400** voucher lỗi

35 • List My Orders `GET /api/v1/orders`

Params: `status?`, `page`, `limit`

sql

```
SELECT o.order_id, o.order_code, o.order_status, o.total_amount, o.created_at,
       s.store_name, oi.product_name_snapshot AS first_item
FROM orders o
JOIN stores s ON o.store_id=s.store_id
OUTER APPLY (SELECT TOP 1 product_name_snapshot FROM order_items WHERE order_id
             WHERE o.customer_id=@uid [AND o.order_status=@s]
             ORDER BY o.created_at DESC
```

→ **200**: `{orders[], pagination}`

36 • Get Order Detail `GET /api/v1/orders/:id`

- Verify `order.customer_id = @uid`
- JOIN: orders + order_items + stores + shipments (LEFT)

→ **200**: `{...order, items[], store{name,logo}, shipment?{status,tracking_code}}` → **403** | **404**

37 • Cancel Order `POST /api/v1/orders/:id/cancel`

Body: `{cancel_reason?}`

- Verify ownership
- Check `order_status = 'PENDING'` → nếu không: **400** "Không thể hủy đơn đã xử lý"
- `UPDATE orders SET order_status='CANCELLED', cancelled_by_user_id=@uid, cancel_reason=@r, cancelled_at=NOW()`
- INSERT order_status_histories (prev='PENDING', new='CANCELLED')
- Hoàn stock: `UPDATE products SET stock_quantity+=qty` theo từng order_item

→ **200**: `{message:"Đã hủy đơn"}` | **400** | **403** | **404**

GROUP 11: Orders - Seller (38-41)

| Auth: Seller | Lấy `store_id` từ token | Verify `order.store_id = @seller_store_id`

38 • List Store Orders `GET /api/v1/seller/orders`

Params: `status?`, `page`, `limit`

sql

```
SELECT o.order_id, o.order_code, o.order_status, o.total_amount, o.created_at,
       u.full_name AS customer_name
FROM orders o JOIN users u ON o.customer_id=u.user_id
WHERE o.store_id=@sid [AND o.order_status=@s]
ORDER BY o.created_at DESC
```

→ **200**: {orders[], pagination}

39 • Get Store Order Detail (GET /api/v1/seller/orders/:id)

- Verify (order.store_id = @sid)
- JOIN: order_items, users (customer), shipments (LEFT)

→ **200**: {...order, items[], customer{full_name,phone}, shipment?} → **403** | **404**

40 • Update Order Status (PATCH /api/v1/orders/:id/status)

Body: {new_status, change_note?}

- Verify (order.store_id = @sid)
- Allowed transitions:

current	→ new
PENDING	CONFIRMED
CONFIRMED	PROCESSING
PROCESSING	READY_TO_SHIP

- Không cho Seller cancel hoặc nhảy status
- INSERT order_status_histories
- Nếu (new_status='READY_TO_SHIP'): INSERT shipments ((order_id=@oid, status='PENDING_ASSIGNMENT'))

→ **200**: {order_id, order_status} | **400** transition lỗi | **403** | **404**

41 • Get Order Status History (GET /api/v1/orders/:id/history) | Seller, Admin

- Seller: verify (order.store_id = @sid); Admin: xem tất cả

sql

```
SELECT h.*, u.full_name AS changed_by_name
FROM order_status_histories h
LEFT JOIN users u ON h.changed_by_user_id=u.user_id
WHERE h.order_id=@id ORDER BY h.created_at ASC
```

→ **200**: `{order_id, history:[{previous_status, new_status, changed_by_name?, change_note, created_at}]}`

GROUP 12: Reviews (42-43)

42 • Create Review `POST /api/v1/reviews` | Customer

Body: `{order_item_id, rating(1-5), comment?}`

- Verify `order_items WHERE order_item_id=@oid → order.customer_id = @uid`
- Check `order.order_status = 'COMPLETED'`
- Check chưa review: `UX_reviews_order_item UNIQUE(order_item_id)`
- `product_id` lấy từ `order_items.product_id`

→ **201**: `{review_id, product_id, rating, comment, created_at}` → **400** đơn chưa hoàn thành | **409** đã đánh giá

43 • Hide Review `PATCH /api/v1/reviews/:id/hide` | Admin, Manager

Body: `{reason?}`

- `UPDATE reviews SET status='HIDDEN', hidden_by_user_id=@uid, hidden_at=NOW()`

→ **200**: `{review_id, status:'HIDDEN'}` | **403** | **404**

GROUP 13: Messaging (44-48)

Auth: Customer hoặc Seller tùy API

44 • List My Conversations `GET /api/v1/conversations` | Customer, Seller

- Customer: `WHERE c.customer_id=@uid`
- Seller: `WHERE c.store_id=@seller_store_id`

sql

```
SELECT c.*, last_msg.message_content AS last_message_preview, last_msg.created_at AS last_message_at, s.store_name, s.logo_url, u.full_name AS customer_name, u.avatar_url AS customer_avatar
FROM conversations c
JOIN stores s ON c.store_id=s.store_id
JOIN users u ON c.customer_id=u.user_id
OUTER APPLY (SELECT TOP 1 message_content, created_at FROM messages WHERE conversation_id=c.conversation_id
WHERE [condition] ORDER BY last_msg.created_at DESC
```

→ **200**: `{conversations:[{conversation_id, store|customer info, last_message_preview, last_message_at, status}]}`

45 • Create Conversation `(POST /api/v1/conversations)` | Customer

Body: `{store_id, order_id?}`

- Check store tồn tại và ACTIVE
- Nếu đã tồn tại (unique constraint): trả **200** với conversation hiện tại
- Chưa có: INSERT conversations

→ **201** (mới) hoặc **200** (đã có): conversation object

46 • Get Messages `(GET /api/v1/conversations/:id/messages)` | Customer, Seller

Params: `page`, `limit`

- Verify participant: `c.customer_id=@uid` OR `c.store_id=@seller_store_id`

sql

```
SELECT m.*, u.full_name AS sender_name, u.avatar_url AS sender_avatar
FROM messages m JOIN users u ON m.sender_user_id=u.user_id
WHERE m.conversation_id=@cid ORDER BY m.created_at DESC
```

→ **200**: `{messages:[{message_id, sender{user_id,full_name,avatar_url}, message_content, is_read, created_at}], pagination}`

47 • Send Message `(POST /api/v1/conversations/:id/messages)` | Customer, Seller

Body: `{message_content}`

- Verify participant
- Validate content không rỗng/whitespace

- INSERT messages
- `UPDATE conversations SET last_message_at=NOW()`
- Emit WebSocket `conversation:{id}:new_message` (nếu WS đã setup)

→ **201**: message object

48 • Mark As Read `PATCH /api/v1/conversations/:id/read` | Customer, Seller

- Verify participant
- `UPDATE messages SET is_read=1, read_at=NOW() WHERE conversation_id=@cid AND sender_user_id!=@uid AND is_read=0`

→ **200**: `{message:"Đã đánh dấu đã đọc"}`

GROUP 14: Vouchers (49-51)

49 • Check Voucher `GET /api/v1/vouchers/check` | Customer

Params: `code(required)`, `order_amount(required)`, `store_id?`

```
sql

SELECT * FROM vouchers
WHERE voucher_code=@code AND status='ACTIVE'
AND starts_at<=SYSUTCDATETIME() AND ends_at>=SYSUTCDATETIME()
AND deleted_at IS NULL
AND (store_id IS NULL OR store_id=@store_id)
```

- Check `order_amount >= min_order_amount`
- Check `usage_limit IS NULL OR used_count < usage_limit`
- Check per-customer: `COUNT(*) FROM order_vouchers JOIN orders ON... WHERE voucher_id=@vid AND orders.customer_id=@uid`
- Tính discount:
 - PERCENT: `MIN(order_amount * discount_value/100, max_discount_amount ?? ∞)`
 - FIXED: `MIN(discount_value, order_amount)`

→ **200**: `{valid:true, voucher_code, discount_type, discount_value, discount_amount}` →

200: `{valid:false, message:"lý do cụ thể"}`

50 • Create Voucher (POST /api/v1/vouchers) | Admin, Seller

Body: {voucher_code, voucher_name, discount_type(PERCENT|FIXED), discount_value, max_discount_amount?, min_order_amount?, usage_limit?, usage_limit_per_customer?, starts_at, ends_at, store_id?}

- Seller: (store_id) bắt buộc phải là store của họ (không tạo được platform-wide)
- Admin: (store_id=NULL) = platform-wide
- Validate: (ends_at > starts_at); PERCENT: (discount_value <= 100)

→ 201: voucher object | 409 code trùng

51 • Update Voucher Status (PATCH /api/v1/vouchers/:id) | Admin, Seller

Body: {status(ACTIVE|INACTIVE|SUSPENDED)}

- Seller chỉ update voucher của store mình; Admin update tất cả

→ 200: {voucher_id, status} | 403 | 404

GROUP 15: Shipments (52-54)

52 • Track Shipment (GET /api/v1/shipments/:orderId) | Customer

- Verify (order.customer_id = @uid)

```
sql
SELECT s.shipment_id, s.tracking_code, s.shipment_status,
       s.assigned_at, s.picked_up_at, s.delivered_at, s.failed_reason
FROM shipments s JOIN orders o ON s.order_id=o.order_id
WHERE s.order_id=@oid AND o.customer_id=@uid
```

→ 200: shipment object → 404 chưa có shipment (đơn chưa READY_TO_SHIP)

53 • List Assigned Shipments (GET /api/v1/shipper/shipments) | Shipper

Params: (status?), (page), (limit)

```
sql
```

```
SELECT s.*, o.order_code, o.recipient_name, o.shipping_province
FROM shipments s JOIN orders o ON s.order_id=o.order_id
WHERE s.shipper_user_id=@uid [AND s.shipment_status=@s]
ORDER BY s.assigned_at DESC
```

→ 200: {shipments[], pagination}

54 • Update Shipment Status PATCH /api/v1/shipper/shipments/:id/status | Shipper

Body: {new_status, failed_reason?}

- Verify shipment.shipper_user_id = @uid
- Allowed transitions:

current	→ new
ASSIGNED	PICKED_UP
PICKED_UP	SHIPPING
SHIPPING	DELIVERED
SHIPPING	DELIVERY_FAILED

- DELIVERY_FAILED: failed_reason bắt buộc
- Nếu DELIVERED: UPDATE orders SET order_status='COMPLETED', completed_at=NOW() + INSERT status_history
- Nếu DELIVERY_FAILED: UPDATE orders SET order_status='DELIVERY_FAILED' + INSERT status_history
- Cập nhật timestamp tương ứng: picked_up_at, delivered_at

→ 200: {shipment_id, shipment_status} | 400 transition lỗi | 403 | 404

GROUP 16: Admin (55-62)

| Auth: Admin hoặc Manager (trừ chú thích riêng)

55 • List All Users GET /api/v1/admin/users | Admin, Manager

Params: role_code?, status?, keyword?, page, limit

sql

```
SELECT u.user_id, u.email, u.phone, u.full_name, u.status, u.created_at,  
       STRING_AGG(r.role_code, ',') AS roles  
FROM users u  
LEFT JOIN user_roles ur ON ur.user_id=u.user_id AND ur.status='ACTIVE'  
LEFT JOIN roles r ON r.role_id=ur.role_id  
WHERE u.deleted_at IS NULL  
      [AND r.role_code=@rc] [AND u.status=@s] [AND (u.email LIKE @kw OR u.full_name  
GROUP BY u.user_id, u.email, u.phone, u.full_name, u.status, u.created_at  
ORDER BY u.created_at DESC
```

→ **200:** `{users:[{user_id,email,phone,full_name,status,roles[],created_at}], pagination}`

56 • Update User Status `PATCH /api/v1/admin/users/:id/status` | Admin only

Body: `{status(ACTIVE|SUSPENDED)}`

- Không tự suspend mình: `id != token.user_id`

→ **200:** `{user_id, status}` | **400** | **403** | **404**

57 • Assign Role `POST /api/v1/admin/users/:id/roles` | Admin only

Body: `{role_code}`

- Check role tồn tại
- Check chưa có: `UX_user_roles_user_role(user_id, role_id)` → **409**
- INSERT user_roles (`assigned_by_user_id=@uid`, `status='ACTIVE'`)

→ **201:** `{user_id, role_code}` | **409** | **404**

58 • List All Stores `GET /api/v1/admin/stores` | Admin, Manager

Params: `status?`, `keyword?`, `page`, `limit`

sql

```

SELECT s.store_id, s.store_name, s.slug, s.status, s.total_products, s.created_at,
       u.full_name AS owner_name, u.email AS owner_email
FROM stores s
JOIN user_stores us ON us.store_id=s.store_id AND us.store_member_role='OWNER'
JOIN users u ON us.user_id=u.user_id
WHERE s.deleted_at IS NULL [AND s.status=@s] [AND s.store_name LIKE @kw]
ORDER BY s.created_at DESC

```

→ 200: {stores[], pagination}

59 • Update Store Status PATCH /api/v1/admin/stores/:id/status | Admin only

Body: {status(ACTIVE|INACTIVE|SUSPENDED)}

→ 200: {store_id, status} | 403 | 404

60 • List All Orders GET /api/v1/admin/orders | Admin, Manager

Params: {status?}, {store_id?}, {customer_id?}, {from_date?}, {to_date?}, {page}, {limit}

```

sql

SELECT o.order_id, o.order_code, o.order_status, o.total_amount, o.created_at,
       u.full_name AS customer_name, s.store_name
FROM orders o
JOIN users u ON o.customer_id=u.user_id
JOIN stores s ON o.store_id=s.store_id
WHERE 1=1
      [AND o.order_status=@s] [AND o.store_id=@sid] [AND o.customer_id=@cid]
      [AND o.created_at >= @from] [AND o.created_at <= @to]
ORDER BY o.created_at DESC

```

→ 200: {orders[], pagination}

61 • Revenue Report GET /api/v1/admin/revenue | Admin, Manager

Params: {from_date(req)}, {to_date(req)}, {group_by(day|store, def:day)}, {store_id?} Chỉ
tính {order_status='COMPLETED'}

group_by=day:

```

sql

```

```
SELECT CAST(o.created_at AS DATE) AS date,
       COUNT(*) AS order_count, SUM(o.total_amount) AS revenue
FROM orders o
WHERE o.order_status='COMPLETED' AND o.created_at BETWEEN @from AND @to
      [AND o.store_id=@sid]
GROUP BY CAST(o.created_at AS DATE) ORDER BY date ASC
```

group_by=store: GROUP BY (o.store_id, s.store_name)

→ **200**: ({from_date, to_date, group_by, total_revenue, total_orders, data:
[{date|store_name, order_count, revenue}]})

62 • Assign Shipper (POST /api/v1/admin/shipments/:orderId/assign) | Admin only

Body: ({shipper_user_id})

- Check (order.order_status = 'READY_TO_SHIP') → **400** nếu không
- Check user có role_code='SHIPPER' → **400** nếu không
- Lấy shipment theo order_id (đã tạo ở API #40)
- (UPDATE shipments SET shipper_user_id=@sid, shipment_status='ASSIGNED',
assigned_at=NOW())

→ **200**: ({shipment_id, shipper_user_id, shipment_status:'ASSIGNED'}) | **400** | **404**

CHECKLIST

- ☐ Tất cả 42 route đăng ký đúng index router
- ☐ Auth middleware đúng role từng API
- ☐ Transaction cho API #34 (Place Order)
- ☐ Parameterized query (không nối chuỗi SQL)
- ☐ Partial update: chỉ SET field có trong request body
- ☐ Status transition đúng (Orders #40, Shipments #54)
- ☐ Response format nhất quán với Groups 1-6